Predicting Airbnb Listing Price Across Different Cities

UE24CS352A Machine Learning Mini Project
Sakaray Vishal (PES1UG24AM241) and Sreedharala Praneetha (PES1UG24AM281)
Code: https://github.com/vxs01/Airbnb-Price-Prediction-MLMini

1. Problem statement

Airbnb hosts usually set prices by guessing, which is hard for new hosts. We predict the nightly price of a listing from its details (size, location, reviews, amenities, description) by re-implementing and improving the CS229 report "Predicting Airbnb Listing Price Across Different Cities" (Luo, Zhou and Zhou, 2019). There are two tasks: **single city** (train and test inside Berlin, Paris or NYC) and **cross city** (train on NYC, Paris or both, test on Berlin, a city the model never sees).

2. Dataset

Inside Airbnb detailed listing files (about 80–90 columns each). As in the paper, listings without a price are removed, only prices of 500 or less per night are kept, and the target is log(price).

City	Scrape	Rows in file	After cleaning	Median price	Features (single city)
Berlin	Sep 2025	14,274	9,133	103 EUR	561
Paris	Mar 2026	38,075	30,971	203 EUR	385
NYC	Jun 2026	30,259	19,148	156 USD	518

3. Approach

Features (the paper's four groups, all fitted on the training split only): C, 25 numeric columns plus latitude/longitude and distance to the city centre (median-filled, clipped to the training 1st–99th percentile, standardised); O, one-hot categories (room/property type, neighbourhood, host flags) plus multi-hot amenities present in at least 1% of listings; T, TF-IDF (unigrams and bigrams) reduced to 50 SVD components per text field; D, dates as days since 2008. Split: 70% train, 20% dev, 10% test, repeated for 3 random seeds.

Models: Ridge linear regression, KNN (k = 5), Random Forest (100 trees), XGBoost (100 trees, depth 6), a neural network (128-512-64, Adam, lr 1e-4) and our addition, a **tuned XGBoost** (lr 0.03, subsampling, early stopping on the dev set). Cross city uses the paper's small network (64-32, SGD with Nesterov momentum 0.9, lr 0.005, L2 0.005, early stopping) and XGBoost on the C+O features, without neighbourhoods.

Improvements over our first version. (1) Our first notebook z-scored every column, including 0/1 one-hot columns. A category that is rare in NYC became a value of about 100 for Berlin, which made the cross-city network predict nonsense (R^2 of −8,244). Now only real-valued columns are scaled. (2) The raw files contain sentinel values such as maximum_nights = 2,147,483,647, so continuous columns are clipped. (3) TF-IDF, medians and vocabularies were fitted before the split (leakage); now they are fitted on the training data only. (4) We use 3 seeds instead of one split. (5) **City calibration:** the model learns the price relative to each training city's median, and is then mapped onto Berlin with a linear fit on only 100 labelled Berlin listings (about 1% of Berlin). The remaining 9,033 Berlin listings are used for testing.

4. Implementation

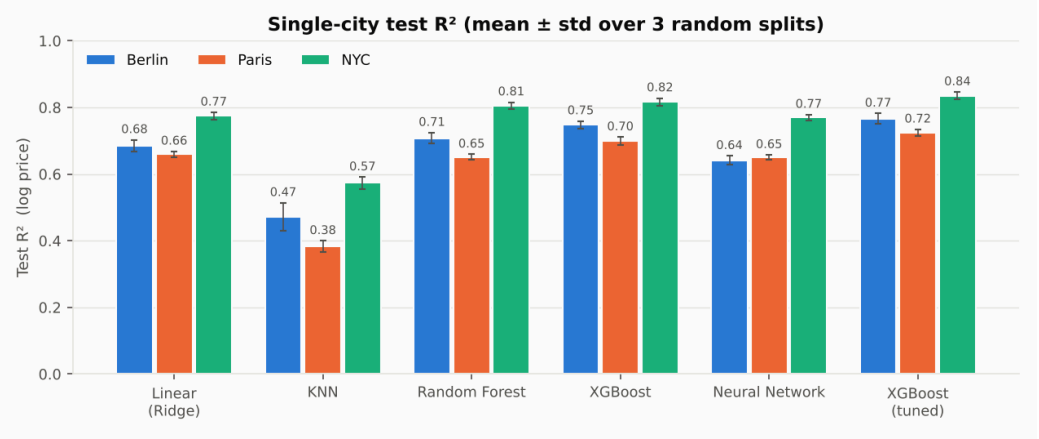
Python package *src/* (pandas, scikit-learn, XGBoost, matplotlib): *data.py* (load, clean, split), *features.py* (FeatureBuilder for C/O/T/D), *models.py*, *single_city.py* and *cross_city.py* (experiments that save every result to *results/*.csv* and every chart to *results/figures/*), *demo.py* (live demo) and pytest smoke tests. We report MSE and R^2 on log price (R^2 of 1 is perfect, 0 is no better than predicting the average) and the mean absolute error in currency units.

5. Results

Test R^2 (mean ± std, 3 seeds)	Berlin	Paris	NYC
Linear (Ridge)	0.685 ± 0.018	0.659 ± 0.009	0.775 ± 0.011
KNN	0.472 ± 0.042	0.383 ± 0.017	0.573 ± 0.017
Random Forest	0.708 ± 0.017	0.651 ± 0.008	0.805 ± 0.011
XGBoost	0.748 ± 0.011	0.699 ± 0.012	0.817 ± 0.011

Neural network	0.641 ± 0.013	0.650 ± 0.007	0.769 ± 0.009
XGBoost (tuned)	**0.766 ± 0.016**	**0.724 ± 0.010**	**0.835 ± 0.011**
Mean abs. error, tuned XGBoost	25 / night	41 / night	35 / night

Table 1: single-city test results. In our first notebook the neural network had R^2 of 0.078 / 0.509 / 0.105 and XGBoost 0.712 / 0.723 / 0.811.



The tuned XGBoost is the best model in every city, and Random Forest and the default XGBoost come next. Fixing the scaling raised the neural network from R^2 0.08 to 0.64 in Berlin and from 0.10 to 0.77 in NYC. Ablations with XGBoost show that size (accommodates, bedrooms), location and room type carry most of the signal. Text adds little once the C+O features are present, and dates alone are almost useless (R^2 0.00–0.14). On the actual price scale, the label transforms (none, square root, log) are within 0.03 R^2 of each other.

Berlin R^2 (trained on)	**First notebook NN**	**NN, scaling fixed**	**NN + calibration**	**XGBoost + calibration**
NYC	−8,243.98	−0.497	0.333	0.266
Paris	−88.30	0.290	0.426	0.468
NYC + Paris	0.088	0.259	**0.581**	0.531

Table 2: transfer to unseen Berlin (mean of 3 seeds). With calibration, the NYC + Paris network's mean absolute error on Berlin is about 35 per night.

6. Conclusions

- Inside one city, tuned XGBoost is the best choice: R^2 0.766 (Berlin), 0.724 (Paris) and 0.835 (NYC).
- The very negative cross-city R^2 values of our first version were caused by scaling 0/1 columns, not by the data.
- Even with the fix, a zero-shot model over-predicts Berlin, because Berlin is much cheaper than Paris and NYC and no listing feature tells the model which city it is in.
- Calibrating with just 100 Berlin prices makes the model transfer well. Training on two cities transfers better than training on one (R^2 0.581 vs 0.333 / 0.426), which agrees with the paper (0.514) and is higher than its value.

Limitations. Prices are in different currencies (EUR, USD), the scrapes are from different months, and the paper used older 2019 Kaggle files, so our numbers are not directly comparable to the paper's. **Future work:** more training cities and a learned city embedding.

Reference: Y. Luo, X. Zhou, Y. Zhou, "Predicting Airbnb Listing Price Across Different Cities", CS229, Stanford, 2019.